

# Real-Time Financial Fraud Detection Pipeline

*Individual Contribution and Technical Delivery Report*

**Kapish Gupta**

Team Lead and Primary Implementation Contributor

|                         |                                                                                                                           |
|-------------------------|---------------------------------------------------------------------------------------------------------------------------|
| <b>Project</b>          | Project 2: Real-Time Financial Fraud Detection Pipeline                                                                   |
| <b>Organization</b>     | Zaalima Development                                                                                                       |
| <b>Dataset</b>          | PaySim Synthetic Mobile Money Transactions                                                                                |
| <b>Repository</b>       | <a href="https://github.com/Kapish196258/real_time_fraud_detection">github.com/Kapish196258/real_time_fraud_detection</a> |
| <b>Reporting Period</b> | July 2026                                                                                                                 |

## Table of Contents

| S.No. | Section                                             | Page |
|-------|-----------------------------------------------------|------|
| 1     | Executive Summary                                   | 3    |
| 2     | Project Scope and Team Context                      | 3    |
| 3     | Role as Team Lead                                   | 3    |
| 4     | Development Approach and Delivery Sequence          | 3    |
| 5     | Data Understanding and Data Quality Work            | 4    |
| 6     | Feature Engineering and Model Development           | 4    |
| 7     | Kafka and Docker Infrastructure                     | 4    |
| 8     | Real-Time Prediction Flow                           | 5    |
| 9     | Spark Structured Streaming Pipeline                 | 5    |
| 10    | MongoDB Persistence and Validation                  | 5    |
| 11    | Prometheus Monitoring                               | 6    |
| 12    | Grafana Dashboards and Validation                   | 6    |
| 13    | Repeatable Load Testing and Producer Range Controls | 6    |
| 14    | Testing and Quality Assurance                       | 6    |
| 15    | Contribution Timeline Based on Git History          | 7    |
| 16    | Key Results and Business Interpretation             | 7    |
| 17    | Repository and Documentation Organization           | 8    |
| 18    | Team Leadership and Collaboration                   | 8    |
| 19    | Limitations and Recommended Next Steps              | 9    |
| 20    | Conclusion                                          | 9    |
| 21    | Screenshot Appendix                                 | 9    |

## 1. Executive Summary

This report documents my individual contribution as Team Lead for the Real-Time Financial Fraud Detection Pipeline. The project converts a large PaySim transaction dataset into a working real-time fraud monitoring system. Transactions are streamed through Kafka, scored inside Spark Structured Streaming, stored in MongoDB, measured through Prometheus, and presented in Grafana dashboards.

My contribution covered the full technical lifecycle: project setup, data analysis, feature engineering, model training, Kafka producer and consumer development, Spark micro-batch processing, MongoDB integration, monitoring metrics, dashboards, testing, validation reports, and final technical coordination. The Git history shows a continuous implementation path from the initial repository structure to the final model-quality dashboard and validation evidence.

## 2. Project Scope and Team Context

The project was completed by a two-member team. I served as Team Lead and handled the main implementation path. The second team member supported model experimentation, documentation updates, Grafana provisioning, and the performance dashboard. The final system is a shared project, while this report focuses on the work directly delivered through my commits and technical decisions.

The original brief proposed Kafka, Spark, a NoSQL database, Prometheus, Grafana, and an anomaly-detection model. The completed implementation uses Python, Kafka, PySpark Structured Streaming, MongoDB, Prometheus, Grafana, and a Random Forest classifier. Python package versions are maintained in the project file requirements.txt, which should be consulted for the exact reproducible environment.

## 3. Role as Team Lead

- Planned the technical sequence and repository structure.
- Implemented the core data, model, streaming, database, monitoring, and dashboard components.
- Created validation reports after major milestones so each stage could be reviewed independently.
- Coordinated integration work and protected the main branch through staged commits and feature branches.
- Validated teammate contributions before merging them into the shared pipeline.
- Prepared final evidence, load tests, and project documentation for review.

## 4. Development Approach and Delivery Sequence

The system was built in the same order in which data travels through the final pipeline. This reduced integration risk because every stage had a working input and a measurable output before the next stage was added.

1. Initialize the repository, folders, environment files, and empty tracked directories.
2. Understand the PaySim schema, transaction types, fraud distribution, missing values, and balance behavior.
3. Clean and engineer reusable fraud-related features.
4. Train and compare models, select Random Forest, and save the model and feature metadata.

5. Create Docker services for Kafka and MongoDB.
6. Build a chunk-based Kafka producer and a Python prediction consumer.
7. Create the Spark Structured Streaming consumer and then extend it to foreachBatch prediction.
8. Add MongoDB batch upserts, prediction history, fraud alerts, and validation queries.
9. Add Prometheus counters, gauges, and histograms to the Spark pipeline.
10. Provision Grafana and build overview, performance, and model-quality dashboards.
11. Add repeatable producer ranges, automated tests, load validation, and final reports.

## 5. Data Understanding and Data Quality Work

The PaySim dataset contains more than 6.36 million simulated mobile-money transactions. Initial analysis confirmed that fraud is rare and appears only in TRANSFER and CASH\_OUT transactions. This class imbalance made false negatives, recall, precision, and F1 score more important than accuracy alone.

- Validated dataset dimensions, schema, transaction types, and class distribution.
- Checked missing values and duplicate records.
- Reviewed origin and destination balance fields for inconsistencies.
- Identified the need for chunk-based reading because loading the full file repeatedly would be inefficient.
- Preserved the original fraud label for later real-time model-quality measurement.

## 6. Feature Engineering and Model Development

Feature engineering was implemented both in notebooks for analysis and in reusable Python code for consistency. Transaction type, amount behavior, and balance relationships were transformed into model-ready inputs. The same ordered feature list is saved with the model so live predictions use exactly the same input structure as training.

### 6.1 Model selection

Multiple model results were compared. Random Forest was selected because it produced a strong balance of recall, precision, and false-negative control for the highly imbalanced fraud problem.

| Metric          | Final evaluation result |
|-----------------|-------------------------|
| Accuracy        | 0.999997                |
| Precision       | 1.000000                |
| Recall          | 0.997565                |
| F1 score        | 0.998781                |
| False positives | 0                       |
| False negatives | 4                       |

## 7. Kafka and Docker Infrastructure

Docker Compose was used to create a repeatable local environment. Kafka receives incoming transactions, MongoDB stores predictions, Prometheus collects application metrics, and Grafana

visualizes the results. Kafka UI and Mongo Express provide operational inspection during development.

The producer reads the PaySim CSV in chunks rather than loading the entire dataset into memory. A chunk size of 50,000 rows allows the script to skip earlier chunks and stream only the requested section efficiently.

```
docker compose -f .\docker\docker-compose.yml up -d
docker compose -f .\docker\docker-compose.yml ps
```

## 8. Real-Time Prediction Flow

The first real-time version used a Python Kafka consumer. Each JSON message was converted into the expected feature structure, scored by the saved Random Forest model, written to MongoDB, and printed for validation. This stage proved that the serialized model could be used outside the training notebook before Spark was introduced.

The prediction helper isolates feature preparation and model loading from the streaming code. This separation prevents training logic from being duplicated across the Python consumer and Spark pipeline.

## 9. Spark Structured Streaming Pipeline

Spark Structured Streaming was introduced in two stages. First, a consumer verified that Spark could parse Kafka messages and form micro-batches. The final pipeline then used `foreachBatch()` to process each batch with normal Python and database logic.

- Parse Kafka JSON messages using a fixed schema.
- Preserve `transaction_id`, stream timestamp, and actual `isFraud` label.
- Convert batch rows into prediction-ready records.
- Apply the Random Forest model and probability calculation.
- Separate valid predictions from failures.
- Upsert prediction history and predicted fraud alerts into MongoDB.
- Record batch, prediction, database, and model-quality metrics.
- Print a readable batch summary for testing and evidence.

```
python -m src.streaming.spark_fraud_pipeline --reset-checkpoint
```

## 10. MongoDB Persistence and Validation

MongoDB was selected as the high-write NoSQL store. The database contains `prediction_history` for all scored transactions and `fraud_alerts` for records where prediction equals 1. Transaction IDs are indexed and used as upsert keys, so rerunning a known range updates existing documents instead of creating uncontrolled duplicates.

- Batch upsert utilities reduce one-document-at-a-time overhead.
- Prediction operations, inserts, and matches are tracked separately.
- Fraud alert operations use the same repeatable transaction identifier.
- Query utilities support collection counts, fraud review, false positives, and false negatives.

## 11. Prometheus Monitoring

Prometheus metrics were added directly to the Spark process and exposed on port 8000. The Prometheus Docker service scrapes this endpoint and stores the time series used by Grafana.

- Processed transactions and predicted fraud alerts
- Prediction failures
- Completed, empty, and failed Spark batches
- Latest batch size, processing duration, and throughput
- MongoDB prediction and alert operations
- True positives, true negatives, false positives, and false negatives
- Batch-duration histogram

## 12. Grafana Dashboards and Validation

Three provisioned Grafana dashboards were delivered. The Overview dashboard provides system-level KPIs. The Performance dashboard focuses on Spark and MongoDB behavior. The Model Quality dashboard converts cumulative confusion-matrix counters into precision, recall, F1 score, and accuracy.

- Fraud Detection Overview - transaction volume, predicted fraud, fraud rate, failures, throughput, duration, and Spark batch status.
- Fraud Pipeline Performance - batch duration, throughput, batch failures, prediction failures, and MongoDB write operations.
- Fraud Model Quality - TP, TN, FP, FN, precision, recall, F1, accuracy, actual fraud, predicted fraud, and classification outcomes over time.

## 13. Repeatable Load Testing and Producer Range Controls

Repeatable testing required a way to avoid reusing the same transaction identifiers. I added `--start-row` so a test can begin at a known zero-based dataset row. The generated `transaction_id` is based on the original dataset position, not the position inside the current chunk or test run.

```
python .\src\streaming\producer.py `
    --start-row 8000 `
    --limit 2000 `
    --sleep 0.01
```

This command reads from row 8000 and generates transaction IDs 8001 through 10000. The `--limit` option controls the total number of records, while `--sleep` controls the delay between messages. Using a new start row for each load test keeps MongoDB validation and dashboard counts understandable.

The producer processes the CSV in 50,000-row chunks. It skips complete chunks before the requested start row, slices the first relevant chunk at the correct local offset, and stops when the requested limit is reached.

## 14. Testing and Quality Assurance

Automated producer-range tests were added with pytest. The suite validates argument handling, start-row behavior, deterministic IDs, limits, empty ranges, and non-overlapping repeated runs. Seventeen tests passed in the validated environment.

- JSON dashboard files were parsed before provisioning.
- Grafana health and dashboard provisioning were checked after restart.
- Prometheus metrics were read directly from the application endpoint.
- MongoDB counts and transaction ranges were queried after Spark completion.
- Feature branches were used for dashboard work before merging to main.
- Runtime folders such as checkpoints, caches, and virtual environments were excluded from commits.

## 15. Contribution Timeline Based on Git History

The following timeline summarizes my implementation path. It is based on the commit history supplied for this report.

| Period     | Delivered work                                                                                                                                               |
|------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| July 2-4   | Initialized the repository, tracked empty folders, completed initial data understanding and EDA, created Docker Compose for Kafka and MongoDB.               |
| July 5-6   | Added the Kafka producer, reusable feature engineering, final model-training analysis, and the model-training pipeline script.                               |
| July 7-9   | Added prediction helper, MongoDB utilities, Python fraud consumer, real-time flow documentation, end-to-end tests, and MongoDB validation queries.           |
| July 10-12 | Added Spark consumer, checkpoint exclusions, foreachBatch processing, Spark prediction, MongoDB batch upserts, and persistence reports.                      |
| July 13-14 | Integrated MongoDB and Prometheus into the Spark pipeline, added Prometheus service configuration, and built the Fraud Overview dashboard.                   |
| July 15-17 | Added repeatable start-row testing, producer range tests, performance validation report, Model Quality dashboard, and final model-quality validation report. |

## 16. Key Results and Business Interpretation

A 2,000-transaction validation run completed without prediction failures. The final large micro-batch contained 1,999 records, including two actual fraud transactions. Both were detected, giving two true positives, 1,997 true negatives, zero false positives, and zero false negatives for that batch.

- The model can detect known fraud patterns with very low false-negative risk in the validated range.
- Zero false positives in the test range avoids unnecessary customer blocking for those transactions.
- MongoDB separates full prediction history from actionable fraud alerts.
- Prometheus and Grafana make pipeline failures and latency visible instead of hiding them in terminal logs.
- Repeatable transaction ranges make demonstrations and regression checks auditable.

## 17. Repository and Documentation Organization

The repository separates experimentation, reusable code, infrastructure, monitoring, tests, and detailed evidence. The structure is intentionally summarized here rather than listing every file again.

- notebooks/ - data understanding, EDA, feature engineering, and model-training notebooks.
- src/ - reusable Python modules for preprocessing, features, models, Kafka, Spark, MongoDB, and monitoring.
- docker/ - Docker Compose services for the local platform.
- monitoring/ - Prometheus configuration, Grafana provisioning, and dashboard JSON files.
- tests/ - automated producer range validation.
- reports/ - milestone reports, troubleshooting notes, and validation evidence.
- models/saved\_model/ - serialized model, feature list, and metadata.
- data/ - raw input, processed files, and generated prediction exports.

## 18. Team Leadership and Collaboration

As Team Lead, I maintained the main implementation path while the second team member contributed selected model, documentation, and Grafana tasks. Shared work was integrated through Git branches and pull requests. The teammate performance dashboard was reviewed through a feature branch, merged through a pull request, and then validated through a separate report.

The final README and screenshot evidence may include contributions from both members. Individual screenshots should be added by the person responsible for the related dashboard, while the shared repository should present one coherent final system.



## 19. Limitations and Recommended Next Steps

- PaySim is synthetic and does not represent every real banking fraud pattern.
- Prometheus counters reset when the local Spark process restarts.
- The validated local pipeline does not prove production sub-second latency at institutional scale.
- Model retraining and concept-drift detection are not automated.
- The project does not currently include a REST API because Kafka is the intended ingestion interface.
- A production extension should add authentication, distributed Spark, Kafka partition tuning, alert notifications, and a model registry.

## 20. Conclusion

My contribution delivered the primary end-to-end technical implementation of the project. The system progressed from a raw six-million-row dataset to a repeatable real-time fraud pipeline with persisted predictions, operational monitoring, model-quality analytics, automated tests, and review evidence. The completed work demonstrates both machine-learning capability and practical streaming-system engineering.

## 21. Screenshot Appendix

The screenshots belong at the end because dashboards were the final delivery stage. They should be inserted after the written implementation and results so the reader first understands what the panels represent.

Figure 1. Fraud Detection Overview dashboard

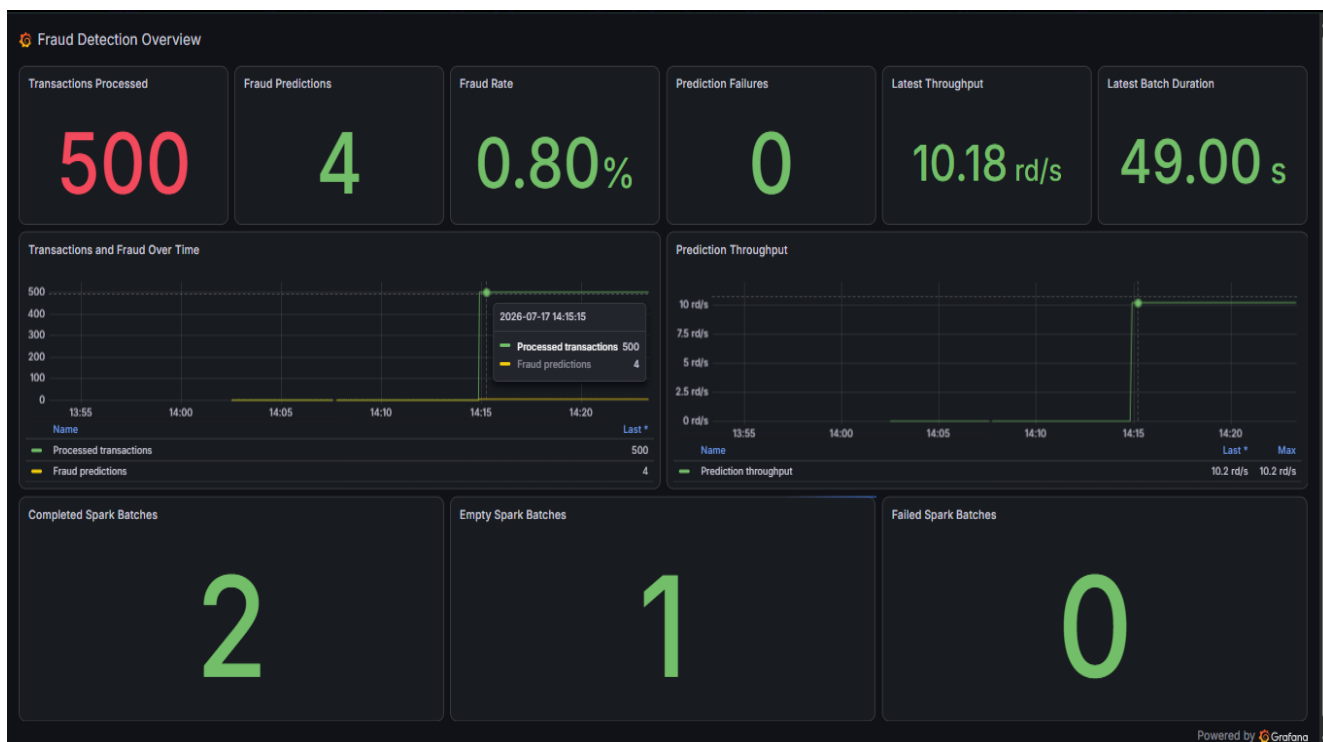


Figure 2. Fraud Model Quality dashboard

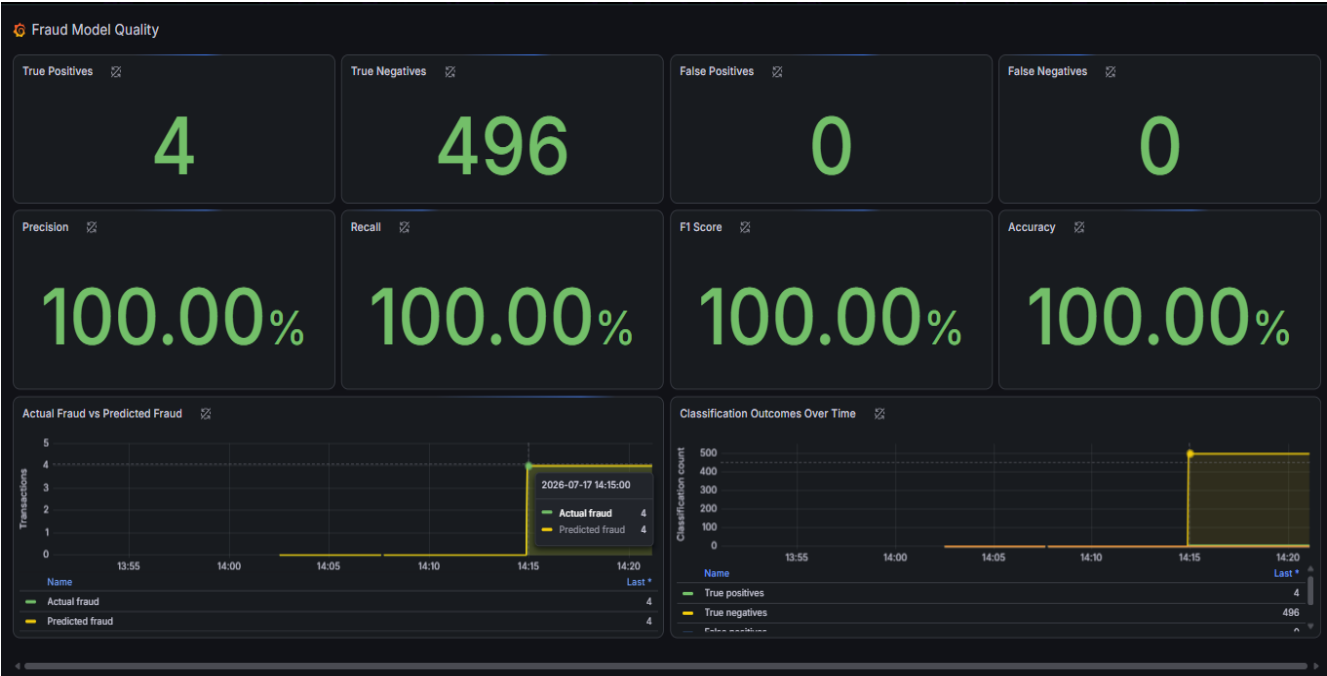


Figure 2. Fraud Pipeline performance dashboard

